Foundations in Data Engineering

Basic building blocks

Jana Giceva



The Basic Problem

- Someone hands you a large dataset. How do you start analyzing it?

- In particular:
  - How do you get a first impression of the data?
  - How do you pick the most appropriate tool for the task?
  - How do you decide about the required hardware?

Learning objectives

By the end of this lecture, you should be able to:

- Compare common data exchange formats by structure, verbosity, and how suitable each is for data.
- Recognize common JSON or CSV errors/pitfalls.
- Explain UNIX stream I/O (stdin, stdout, stderr) and use pipes and redirection to combine commands.
- Inspect large or compressed text files without decompressing them to disk.
- Write regular expressions with grep to filter, invert and extract data.
- Use command line tools as relational operators.
- Build multi-stage pipelines.
- Evaluate the limits of shell-based analysis.

File formats

high level overview

File formats

- As we will discuss later in class
  datasets can be exchanged using many formats.

- Some popular choices:
  - Text files, CSV files
  - JSON
  - XML
  - Binary but open formats like Parquet
  - Proprietary binary formats (not well suited for exchange)

- When in doubt, `file` can be a useful tool.

CSV Files

A plain text file contains rows of data (comma separated values):

```
Sherlock Holmes,221B Baker Street,Detective
James Moriarty,Reichenbach Falls,Villain
```

- Simple, human readable formats
- Very popular, widely used
- But a lot of subtleties
- Separator customizable (*e.g.*, tab separated values (TSV))
- String with separator require quoting
  - `“string with “” inside”`

XML Files

- A text format encoding (semi-)structured data:

```
<characters>
<person name="Sherlock Holmes" address="221B Baker Street" job="Detective"/>
<person name="James Moriarty" address="Reichenbach Falls" job="Villain"/>
</characters>
```

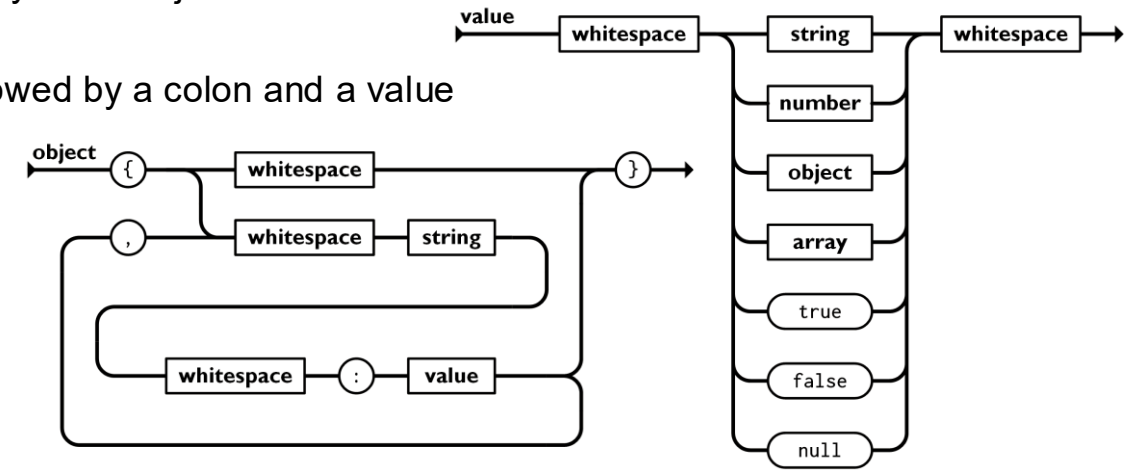
- Format itself better standardized than CSV
- Suitable for nested data, too
- Human readable, but not very friendly to write
- Very verbose, also full XML spec is very complex
- Allows for advanced features (e.g., XML schema), but these are rarely used
- Somewhat popular, but declining

JSON Files

- A text format stemming from JavaScript Object Notation:

```
[{"name":"Sherlock Holmes", "address":"221B Baker Street", "job":"Detective"},
 {"name":"James Moriarty", "address":"Reichenbach Falls", "job":"Villain"}]
```

- JSON always starts with either an array or an object.
- Much simpler format than XML
  - keys must be in double quotes, followed by a colon and a value
  - only 6 data types
- Less verbose and easier to write
  - items separated by commas
  - no trailing commas
  - allows nesting
- Growing popularity



Exercise

- Let’s debug this JSON together

```
{
  name: ‘Ada’,
  "age": 28,
  "skills": ["math", "logic"],
}
```

- What are the three mistakes?

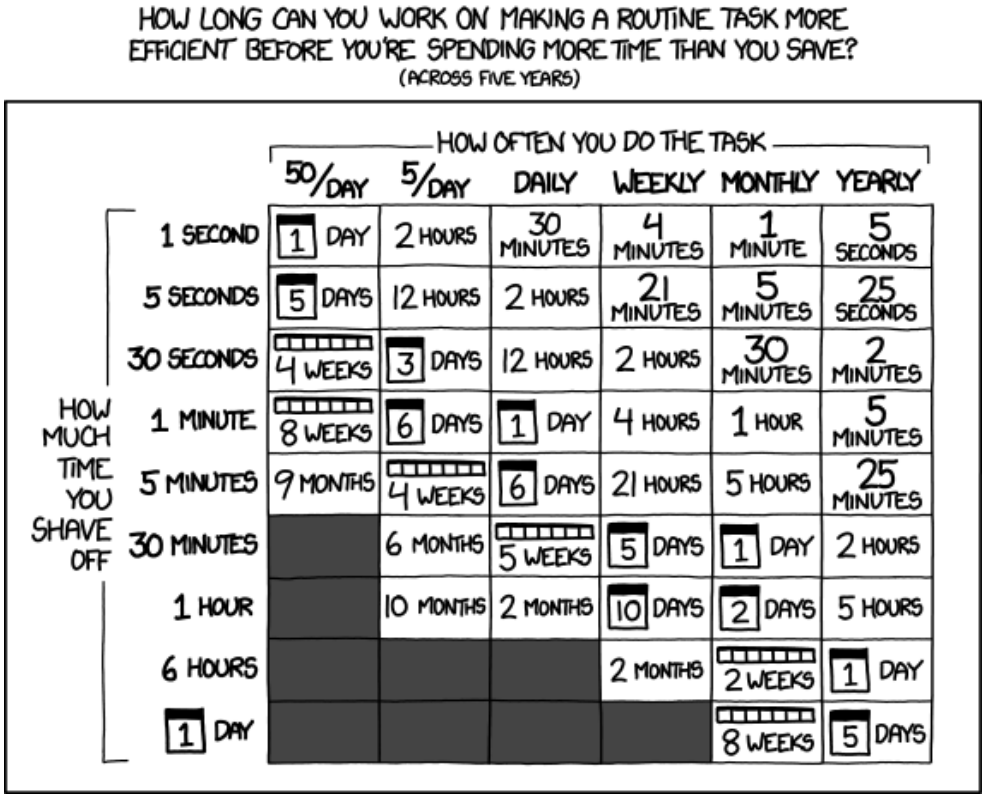
Command Line Tools

Working with Text Files

- Text files are quite popular because they are human readable and easy to use.

- We start with **command line tools**
  - Available by default on Unix/Linux platforms
  - Simple, but surprisingly **powerful in combination**
  - Ideal to get a **first impression**
  - Allows to examine and explore the data
  - Ultimately we want to pick a better tool
  - But command line tools allow for simple analysis without any code

Is it worth the time?



http://xkcd.com/1205/

CC BY-NC 2.5

Combining Tools

- Many tools can read and write files, but it is often more convenient to **combine them with pipes**.
  - Every program has an **input stream** and an **output stream**
  - Piping both;
    `command1 | command2`
  - Redirecting to a file:
    `command > file`
  - Redirecting from a file:
    `command < file`
  - In addition: error stream
    `command 2> /dev/null`

More ways to combine commands will be covered in the exercises

Unix stream IDs:
0 – stdin
1 – stdout
2 – stderr

Suppress the error stream of "command" and redirect it to the null device (/dev/null), which ignores its input (like a black hole)

Reading Files - `cat`

- Reading a single file
  `cat file`
- Concatenating multiple files
  `cat file1 file2 file3`
- Accepting piped input
  `command | cat`
- Escaping binary data
  `cat -v binaryfile`
- For compressed files
  `zcat / bzcat / xzcat`

`zcat` is identical to `gunzip -c` (Lempel-Ziv)
`bzcat` is identical to `bunzip2` (Burrows-Wheeler Transforms)
`xzcat` is identical to `unxz` (Lempel-Ziv-Markov v2)

- Mainly used as input for other commands

Paging results – `less`

- Used to inspect results or files, gives the results page by page and can be scrolled back and forth between pages.
- In pipelines mainly used as the last command
- Paging a command result
  ```
  command | less
  ```
- Paging a file
  ```
  less file
  ```
- Chopping long lines
  ```
  less -S
  ```

Filtering – grep

- Returns all qualifying lines – very powerful tool with many options
- Filtering input

```
cat file1 file2 | grep 'img[0-9]*\.jpg'
```

- Filtering files

```
grep 'img[0-9]*\.jpg' *.html
```

- Returning only the match

```
command | grep -o 'user=.*$'
```

- Returning only non-matching lines

```
grep -v '^warning' logfile
```

- Case insensitive search

```
grep -i '\.jpg' file
```

Good time to refresh the basics for reg-expressions:
Q: What does * stand for? What about +?
Q: What is $ used for? What about ^?
Q: How about '?'?

Examples

Let's try to make things more interactive.

- Go to Moodle and download the LKML dataset (gmane.linux.kernel.xz)
- The dataset is in compressed form (need `xzcat` to look into it).

How would you do the following tasks. Work in pairs. You have 5 min.
There are *multiple ways* to solve them – there is no one *correct* solution.

- Find all senders on the LKML

- Find all senders that are not using .com domains

- Extract the email part of the sender address
  - *Hint:* RFC standard regular expression for email
    https://gist.github.com/baker-ling/3b4b014ee809aa9732f9873fe060c098#the-pattern

Discussion

- Find all senders on the LKML
  - How do you work with a compressed format?
  - How do you make `grep` work with a binary?
  - How do you filter the senders?
- Find all senders that are not using .com domains
  - How do you do inverse search?
  - How do you filter the '.com' domains?
- Extract the email part of the sender address
  - How do we extract only the email part?
  - How do we parse the email address?

Sorting – sort

- Sorts the input according to a criterion
  - Basic sorting

    ```
    cat file | sort
    ```

  - Sorting numerically

    ```
    cat file | sort -n
    ```

  - Sorting a specific key

    ```
    cat file | sort -k 2
    ```

  - Sorting with explicit separator

    ```
    cat file | sort -t ',' -k 3
    ```

  - Sorting, eliminating duplicates

    ```
    cat file | sort -u
    ```

- Very powerful and useful
- Handles files larger than main memory
- Sorted files used as inputs for other algorithms

By default, `sort` sorts elements lexicographically

Selecting Prefix/Suffix – head/tail

- Returns the begin/end of a file
  - First 20 entries
    ```
    sort file | head -20
    ```
  - Last 15 entries
    ```
    sort file | tail -15
    ```
  - Everything but the first two entries
    ```
    sort file | tail –n +3
    ```
  - Everything but the last two entries
    ```
    sort file | head –n -2
    ```

-n + NUM : skip NUM-1 lines at the start, print everything starting with NUM

- All these are top-k queries – sorted then followed by head/tail.
- Also useful for min/max computations.

Handling Duplicates – `uniq`

- Handles duplicates in **sorted** input
  - Eliminating duplicates

    ```
    sort file | uniq
    ```
  - Counting duplicates

    ```
    sort file | uniq -c
    ```
  - Returning duplicates

    ```
    sort file | uniq -d
    ```
  - Returning unique lines

    ```
    sort file | uniq -u
    ```
- Very useful for grouping and counting

Examples

- **Set-up is like before**. Use the same LKML file to do the following tasks.
  - There can be multiple correct solutions.
  - You can also use some of the previous answers.

- **How would you do the following tasks? You have 10min. Work in pairs.**

  - Find all sender emails and sort them alphabetically.

  - Find the 10 most prolific senders to the LKML.

  - Find all senders who wrote only a single mail.

Discussion

- **Find all sender emails and sort them alphabetically.**
  - As before + sort. How do you sort alphabetically?

- **Find the 10 most prolific senders to the LKML.**
  - First find unique senders and count them. How?
  - Then sort them by count, and find the top 10. How?

- **Find all senders who wrote only a single mail.**
  - Sort all senders.
  - Find the unique ones.

Projecting Columns – cut

- Allows to return only relevant parts of the input
  - Return specific fields

    ```
    cat file | cut -f 1,3
    ```

  - Cut with specific delimiter

    ```
    cat file | cut -f 2-4 -d ‘,’
    ```

  - Using characters instead of fields

    ```
    cat file | cut -c 1-10
    ```

Counting - wc

- Counts the number of lines / words / characters
  - Full statistics
    ```
    cat file | wc
    ```
  - Count the number of lines
    ```
    cat file | wc -l
    ```
  - Count the number of bytes
    ```
    cat file | wc -c
    ```
- Useful to gather statistics about the data.

Shuffling – `shuf`

- Shuffles the lines of the input to generate a random permutation
  - Random order

    ```
    cat file | shuf
    ```

- Not so commonly used, but very useful in some cases
  - Obtaining a random sample

    ```
    cat file | shuf | head -10000
    ```

- Result much more useful than without `shuf`
- Also for performance tests

Examples

- **Set-up is like before**. Use the same LKML file to do the following tasks.
  - There can be multiple correct solutions.
  - You can also use some of the previous answers.

How would you do the following tasks?

- Count the number of emails in the archive.
- Pick 20 mail subjects at random.
- Compute the most popular content type.

Discussion

- Count the number of emails in the archive.
- Pick 20 mail subjects at random.
- Compute the most popular content type.

Editing Text – sed

- Replacing one text with another `s/pattern/replacement/g`

  - Replacing one text with another

    ```
    cat file | sed ‘s/oldText/newText/’
    ```

  - Replacing all occurrences of a text

    ```
    cat file | sed ‘s/oldText/newText/g’
    ```

  - Using *back-references*

    ```
    cat file | sed ‘s/IMG_\([0-9]*\).JPG/image\1\.jpg/g’
    ```

  - Case insensitive matching

    ```
    cat file | sed ‘s/file[0-9]*\.png/”&”/I’
    ```

  - Using multiple patterns

    ```
    cat file | sed –e ‘s/old1/new1/’ –e ‘s:a/b:a_b:’
    ```

- Extremely powerful and useful
- Many more features (grouping, conditional branching, etc.), but these are rarely used.

Combining Files – `join`

- Combining sorted input files
  - Combining two files on a common `FIELD`

    ```
    join -1 2 -2 1 file1 file2
    ```
  - Combining with explicit delimiter

    ```
    join -1 2 -2 1 –t ‘,’ file1 file2
    ```
  - Preserving unmatched lines (`-a`);

    ```
    join -1 2 -2 1 –a 1 file1 file2
    ```

`-1 FIELD` – join on this field for `file1`
`-2 FIELD` – join on this field for `file2`

- Behaves like a relational join
- But if you need that it might be better to use more powerful tools

Depends on the ***flag*** used:
`-a 1` (left outer join)
`-a 2` (right outer join)
`-a 1 –a 2` (full outer join)

`-v` suppresses the joined output lines:
`-v 1` (left anti join)
`-v 2` (right anti join)

Computations - awk

- Executes a program for every line of input
  - General structure

    ```
    awk ‘BEGIN { init-code } { per-line-code } END { done-code }’ file
    ```
  - Counting the number of lines

    ```
    awk ‘BEGIN { x=0 } { x=x+1 } END { print x }’
    ```
  - Summing up a column

    ```
    awk ‘BEGIN { x=0 } { x=x+$2 }  END { print x }’
    ```
  - Averaging a column

    ```
    awk ‘BEGIN { x=0; y=0 } { x=x+$2; y=y+1 } END { print x/y }’
    ```
  - Conditions

    ```
    awk ‘BEGIN { x=0 } { if ($1> 10) x=x+$2 } END { print x }’
    ```
- And many other features

Examples

- **Set-up is like before**. Use the same LKML file to do the following tasks.
  - There can be multiple correct solutions.
  - You can also use some of the previous answers.

- **How would you do the following tasks?**

  - Find the most popular words in the LKML data set.

  - Compute the average number of lines in a mail.

  - Find all mail senders who never get a response.

Discussion

- Find the most popular words in the LKML data set.
- Compute the average number of lines in a mail
- Find all mail senders who never get a response.

References

Lecture slides:

- *Foundations in Data Engineering* by Prof. Neumann (TUM)

Command-line tools:

- For more details check the Linux man pages
  - *e.g.*, `grep` command:
    - https://man7.org/linux/man-pages/man1/grep.1.html